LITTERBOXD: A Native Android Application for Daily Cat Care, Health Pattern Tracking, and Memory Keeping

Mobile Computing Group Project Report

natashawa225 · Raymond Harlie · Darren · [Member — to confirm] · farrell

Abstract

Litterboxd is a native Android application for everyday cat care, built in Java on the standard SQLiteOpenHelper and AlarmManager APIs without Room, Compose, or architecture-component libraries. It combines a Flo-style daily check-in with a decision-tree questionnaire, a rule-based health-alert engine that compares recent answers against a historical baseline, auto-generated and user-defined care tasks with scheduled reminders, a cat diary that merges owner-written memories with care history read directly from the weight and check-in tables, and a live "read more" search against a veterinary-information website for alerts that fire. The project was implemented by a five-person team over four weeks on top of an initial onboarding and persistence skeleton, followed by a documented audit-and-fix phase that resolved twenty-four correctness defects before new features were layered on. This report documents the application's actual architecture, traces each feature to its implementing classes, reports the engineering decisions and defects recorded during development, evaluates the application against comparable commercial products (Letterboxd, Flo, and 11pets), and discusses the privacy and ethical considerations raised by storing cat health data and scraping a third-party health website at runtime.

1. Introduction

1.1 Motivation

Caring for a cat well requires remembering several independent things at once: whether it has eaten today, whether its litter box behaviour has changed, when its next vaccination or grooming is due, and whether a symptom noticed last week is a one-off or the start of a pattern. Most of this information currently lives in an owner's memory, in scattered notes, or not at all. Generic habit trackers and note-taking apps can record any of these facts individually, but none of them connect a logged symptom to a question that was already answered three days earlier, and none distinguish "the cat skipped breakfast once" from "the cat has skipped a meal on four of the last seven days." Litterboxd was built to test whether a small set of plain Android components — a SQLite schema, a background executor, and a hand-written decision tree — could support that kind of longitudinal reasoning on a student's course-constrained toolchain, while still being pleasant enough day-to-day that an owner would actually open it every day.

This motivation sits inside a broader, well-studied problem in personal-informatics research: tools that ask people to log something about themselves regularly tend to be abandoned once the effort of logging outweighs the value of what comes back. Li, Dey, and Forlizzi's stage-based model of personal informatics systems describes five stages a tracking tool must support — preparation, collection, integration, reflection, and action — and identifies barriers at each stage, most relevantly that collection becomes a chore when it is not tightly coupled to a fast reflection step that makes the effort feel worthwhile [1]. Rooksby et al.'s interview study of real tracker users similarly found that people rarely rely on a single tool in the disciplined way its designers intended; instead they weave several trackers together loosely, often for short-term, situational reasons rather than a long-term quantified record [2]. Litterboxd's design choices follow directly from both findings: the check-in is answered with chips rather than free text specifically to keep the collection step fast (addressing Li et al.'s first barrier), and the Daily tab surfaces a reflection step — the health-alert banner and Reading Corner — automatically from the same data, rather than requiring a separate "view my history" action, so that the reflection stage Li et al. describe as frequently skipped is instead the default view.

1.2 Existing applications and their gaps

Three products shaped the brief the team was given and are used as reference points throughout this report.

Letterboxd is a film-logging social network: users log a film with the date they watched it, and the app assembles those dated entries into a personal diary that can be revisited and reviewed later [8]. Litterboxd borrows this "log a dated entry, read it back later" idea directly — its own Diary tab is a chronological record of memories and milestones — but there is nothing to log in Letterboxd beyond a single artefact (a film) once per viewing; it has no notion of a recurring daily check-in, no reminders, and no cross-referencing between entries.

Flo is a menstrual-cycle tracker whose core loop is daily logging of a wide set of symptoms, which Flo then turns into personalised cycle insights and a symptom checker for reportable conditions, with some insight features gated behind a paid subscription [9]. Flo demonstrates that a chip-based daily questionnaire can be a sustainable long-term habit at consumer scale, which is the direct model for Litterboxd's own check-in screen. However, Flo's domain is a single, well-understood physiological cycle; it does not need to decide whether an answer is isolated or recurring across an open-ended set of independent symptoms the way a general health-alert engine for an animal must.

11pets is an existing pet-care application whose own marketing describes reminders for appointments, vaccinations, de-worming, and feeding, together with weight and vital-sign tracking and vet record sharing [10]. It is the closest existing

product to Litterboxd's stated goal, and its reminder-and-records model is reflected in Litterboxd's care-tasks feature. Its own listings, however, describe a records-and-reminders tool rather than a day-to-day interaction the owner is drawn back into; nothing in its public description suggests pattern detection across logged days, a diary of memories distinct from medical records, or an engaging per-visit interaction akin to Litterboxd's "pet the cat" touch animation.

1.3 What makes Litterboxd different

Litterboxd's position relative to these three products is not that it does any one of their jobs better, but that it joins three things that are normally separate products: a quick daily log (like Flo), a rule engine that treats consecutive and clustered answers differently from one-off answers (which none of the three above do, since Flo's insights are proprietary and 11pets does not describe pattern detection at all), and a personal diary that mixes the owner's own memories with care history pulled automatically from the same database (which Letterboxd does only for its own single content type, and which 11pets does not appear to do at all). The health-alert engine specifically distinguishes a sudden cluster of a symptom from a sustained shift away from a measured baseline rate, and keeps its wording to "worth watching" / "see a vet" rather than a diagnosis, a distinction discussed further in Section 3.

1.4 Main contributions

This report's contribution is not a claim that Litterboxd is a finished consumer product; it is a description of what was actually implemented, verified against the source code, together with:

- a working decision-tree questionnaire engine with cycle and reachability validation, unit-tested independently of the Android framework;
- a rule-based alert engine supporting both fixed-window signal matching and baseline-ratio comparison against a cat's own recent history, including a weight-trend precondition;
- an automatic care-task generator that keeps feeding reminders synchronised with a cat's feeding schedule without duplicating history when a schedule is edited;
- a documented, fourteen-commit audit-and-remediation pass (Section 3.1) that is itself evidence of the team's engineering process, not only its output.

2. App Design

2.1 Technology stack and constraints

Litterboxd is a single-module Gradle Android application (:app, Kotlin DSL build scripts, compileSdk/targetSdk 33, minSdk 26) written entirely in Java. The project follows a team-maintained technical reference (docs/course_reference.md) limiting the codebase to plain Android APIs taught in the course, with one documented exception (the Glide image-loading library) discussed in Section 3.1(d). The practical effect of that reference on the app's architecture is described in Section 2.2; the only runtime dependencies are Glide, AndroidX AppCompat, Material Components, and ConstraintLayout.

2.2 Architecture

Because architecture-component libraries were off the table, Litterboxd uses a plain three-layer structure rather than MVVM:

- Presentation layer — twenty Activities under ui/, each owning its own XML layout, plus small custom Views (PetTheCatView, WeightChartView, CatCardRow, CatSwitcher) and RecyclerView adapters. Activities read view state on the UI thread, submit work to a shared executor, and post results back to the UI thread themselves; there is no ViewModel or LiveData layer mediating this.
- Domain layer — plain-Java packages with no Android imports, so they can be unit-tested on the JVM without an emulator: checkin (the decision tree and per-session answer logic), health (the rule evaluator, weight-trend helper, and the HTML scraper for "more info" pages), and task/scheduling helpers in data (FeedingTaskPlanner, CareTaskSetup).
- Data layer — LitterboxdDbHelper (a SQLiteOpenHelper singleton [3], obtained through getInstance(context) rather than re-opened per Activity, schema version 7) with a LitterboxdContract of table/column constants and one DAO class per table (CatDao, CareTaskDao, DailyCheckinDao, DiaryDao, FeedingScheduleDao, WeightLogDao), plus FileStorageHelper for photo files in internal storage and AppPrefs for SharedPreferences-backed onboarding/session state.

Several of the audit fixes in Section 3.1(a) are, in Android terms, standard Activity-lifecycle handling that the initial implementation had simply omitted: saving transient UI state (a picked photo, an in-progress feeding-schedule row) in onSaveInstanceState so it survives a configuration change, per the documented Activity lifecycle [7], and declaring POST_NOTIFICATIONS before relying on NotificationManager to post a reminder, per Android's runtime-permission model for notifications [5][6].

Cross-cutting pieces sit alongside these three layers: AppExecutors is a process-wide singleton exposing one fixed thread pool and one main-thread Handler, used everywhere a DAO call must leave the UI thread; the reminders package (CareReminderScheduler, two BroadcastReceivers, and BootReceiver) wraps AlarmManager for both per-task reminders and a daily "today's list" reminder, re-armed on boot, time-zone change, and package replacement; and LitterboxdApplication creates the app's one notification channel at process start so no Activity has to, following the standard pattern of creating a channel once before any notification targets it [5]; the reminders themselves are exact, wake-while-idle alarms scheduled through AlarmManager, the API Android documents for time-sensitive, user-visible reminders [4]. Figure 1 summarises this structure and its one external dependency.

Figure 1. Litterboxd layered architecture

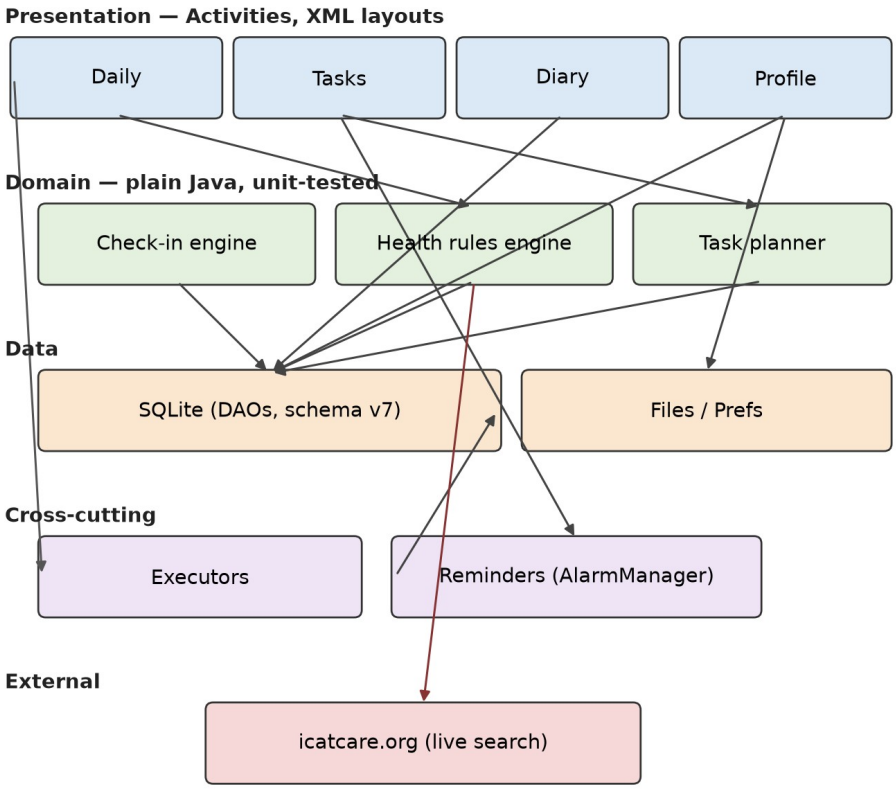


Figure 1. Litterboxd layered architecture, reconstructed from the implementation (Section 2.2).

2.3 Application workflow

A first launch routes SplashActivity → WelcomeActivity (a full-screen, TextureView-based looping cat video wallpaper) → OwnerNameActivity → AddEditCatActivity → FeedingScheduleActivity → HomeActivity, inserting one Cat row and its feeding-schedule rows along the way; AppPrefs records onboarding_complete only once step three finishes, and SplashActivity resumes an interrupted onboarding at the right step rather than restarting it (see Section 3.1). From HomeActivity, a bottom navigation bar (BottomNavHelper) switches between four tabs — Daily (HomeActivity), Tasks (CareTasksActivity), Diary (DiaryActivity), and Profile (OwnerProfileActivity) — each a separate Activity launched with single-top semantics rather than Fragments.

The Daily tab is the app's working surface: it shows the selected cat, today's check-in status, a card-based summary of today's care tasks, any live health alerts, and a horizontally scrolling "Reading Corner" carousel of suggested articles. Tapping "Log today" opens CheckInActivity, a two-screen flow (chip groups, then any follow-up questions the chosen chips unlock) driven entirely by CheckInSession against the DecisionTree loaded once from res/raw/checkin_tree.json; saving writes one daily_checkins row and its checkin_answers rows for that date, or updates them if the cat was already checked in today. HealthLogActivity offers a calendar view of past check-in days and reopens CheckInActivity for any of them. On every save, HomeActivity re-evaluates HealthRuleEvaluator against the answer history and the cat's weight log and refreshes the alert banner and Reading Corner without a separate sync step.

2.4 Main components and class responsibilities

Representative classes, grouped by the feature they implement:

- Check-in (checkin/, ui/checkin/): DecisionTree validates and indexes the question graph (unique ids, single-choice follow-ups, full reachability, no cycles — enforced in its constructor, Section 3.1); CheckInSession tracks the live answer state for one sitting; HistoryTrigger/HistoryTriggerEvaluator inject extra follow-up questions when a chip has been chosen on a run of recent days; TreeJsonParser reads the tree from res/raw/checkin_tree.json. CheckInActivity (486 lines) renders both screens of one session; HealthLogActivity (358 lines) is the calendar launcher.
- Health (health/, ui/health/): HealthRuleEvaluator matches HealthRules read from res/raw/health_rules.json either as fixed-window signal counts ("appetite drop logged on ≥3 of the last 5 days") or as a baseline-ratio comparison between a recent window and a longer prior window, optionally gated by a minimum weight drop computed by WeightTrend; more specific rules can supersede general ones, and dismissed rules stay suppressed. HealthArticleClient/HealthArticleFinder perform a live, read-only HTTP search against icatcare.org for pages

matching an alert's own search terms, validating the response host before use; ReadingCard/HealthContentParser build the Reading Corner feed from res/raw/cat_care_articles.json.

- Care tasks (data/FeedingTaskPlanner.java, data/CareTaskSetup.java, ui/tasks/CareTasksActivity.java): CareTaskSetup keeps each cat's feeding-derived reminders in sync with its feeding schedule and otherwise creates nothing automatically, replacing an earlier six-task generic seeder; FeedingTaskPlanner matches existing reminders to schedule rows by time so an edit updates rather than duplicates a task; CareTasksActivity (628 lines, the largest class in the app) renders to-do/done lists for daily, monthly, and one-off tasks and drives CareReminderScheduler.
- Diary (ui/diary/): DiaryActivity (475 lines) merges owner-written diary_entries (memories and milestones, via DiaryDao) with care history read directly from WeightLogDao and DailyCheckinDao into one newest-first, date-headed list with simple filters; DiaryEntryEditActivity edits a memory, including an optional photo and comma-encoded tag keys (util/DiaryTags); DiaryMemoriesActivity lays out every diary photo as a read-only collage.
- Profile (ui/profile/): CatProfileActivity (599 lines) shows one cat's details, weight history (WeightChartView), and feeding schedule, and hosts the add-weight dialog; ManageCatsActivity and EditProfileActivity handle multi-cat and owner-identity editing; avatars are a fixed illustrated set (CatAvatars, OwnerAvatars), not photo uploads, for the owner's own profile.

2.5 Resources

The resource tree holds 43 Activity/dialog/list-item layouts under res/layout/, five mipmap density buckets for a single adaptive launcher icon, a font/ directory with a bundled display font (title_font.xml, a bold sans-serif-black family used for the welcome and diary headings), and res/raw/ content data: three JSON files driving the check-in tree, health rules, and article feed; sample diary photographs for two seeded demo cats ("Pip" and "Biscuit"); and four looping MP4 clips used by the welcome screen and the "pet the cat" interaction. values-night/themes.xml was deliberately removed during the audit pass (Section 3.1) because an empty night theme was silently discarding the app's brand colours in dark mode.

2.6 Group member contributions

Contributions below are reconstructed from git log --name-only across all 74 commits on main, cross-checked against commit subjects and file ownership rather than inferred from whoever last touched a file. Two pairs of Git identities are treated as one contributor each because their emails and commit timing are consistent with the same person committing under two local configurations; this inference should be confirmed with the team before submission (Table 1).

Table 1. Group member contributions, reconstructed from git log --name-only across all 74 commits on main.

Group Member (Git identity)	Components Developed	Technical Responsibilities	Other Contributions
natashawa225 (51 commits; repository owner)	Project scaffolding WelcomeActivity wallpaper screen Onboarding flow (OwnerName, AddEditCat, FeedingSchedule) HomeActivity dashboard + bottom navigation ui.profile / ui.settings CareTaskSetup / FeedingTaskPlanner refactor Diary seed data and UI polish	Ran and resolved the 24-item audit pass (Sec. 3.1): threading, rotation, validation, navigation bugs Shared-instance LitterboxdDbHelper AgeUtils / Validators / DiaryTags utilities	Authored docs/course_reference.md Wrote AUDIT_REPORT.md and MANUAL_TEST_CHECKLIST.md Pet-the-cat touch interaction, dashboard animation
Raymond Harlie (git: h4rlieee / h4rlieeee, 13 commits combined — same person inferred, confirm)	Daily check-in feature end to end checkin package: DecisionTree, TreeJsonParser, CheckInSession, HistoryTrigger/Evaluator DB schema v4 (daily_checkins, checkin_answers) ui.checkin screens	Health reading suggestions and repeat-day alert triggers Search-failure wording fix	Merged pull requests #1–#4 on GitHub (maintainer role)
Darren (4 commits, large diffs)	Health article search and presentation HealthArticleClient, HealthArticleFinder, SearchedArticle Health calendar, alerts and weight-history screens	Live search integration against icatcare.org Result caching / logging for the search client	—
[GROUP MEMBER — CONTRIBUTION TO BE CONFIRMED] (git: k4chow / kachow, 3 commits combined)	Diary tab (ui.diary): entries, photos, milestones, care history Cat-avatar selection and Manage Cats display fixes	A large "sync current app" commit touching many existing files — may be integration work rather than new authorship (needs confirmation)	—

farrell (3 commits, early in project)	Original daily/monthly care-tasks feature (later superseded by CareTaskSetup / FeedingTaskPlanner)	Early “Update Android project” commit introduced the stray petcare-litterboxd-main/ snapshot (Sec. 4.4)	—

Two further points are flagged for the team to confirm before submission: (1) whether k4chow/kachow is the same person as one of the four names above, and their real name for the table; (2) the exact division of early onboarding/persistence work before the Git history settles into feature branches (the first commit on 2026-10-14 is attributed solely to natashawa225, but it may represent a squashed merge of earlier pair-programmed work).

3. Evaluation

3.1 Development challenges

(a) Correctness debt from rapid early development. Before any of the check-in, health, or diary features existed, an internal audit (AUDIT_REPORT.md, 2026-10-24) against the then-current onboarding/persistence skeleton found twenty-four distinct defects, including a notification permission that could never be granted because POST_NOTIFICATIONS was never declared, an “edit cat” screen that silently inserted a duplicate cat because it never read the cat_id extra, weight values silently clamped to a default when a comma decimal separator was typed, and several Activities reading View state from a background thread after dispatching work to the executor. The team’s chosen response was not a rewrite but a prioritised, one-commit-per-fix remediation branch: fourteen numbered “P1” fixes for crashes and data corruption were merged first, followed by course-constraint and redundancy clean-up. The alternative of silently patching issues inside ongoing feature work was rejected because it would have made the fixes impossible to review or revert individually; the cost of the chosen approach was a full day spent on fixes before any new feature work resumed.

(b) Keeping feeding reminders in sync without duplicating history. An early, simpler design gave every cat the same six fixed care tasks (feeding, litter, play, grooming, vet, weigh-in) regardless of what the owner actually told the app. This was replaced by CareTaskSetup, which derives only feeding reminders from the cat’s own schedule and lets the owner add anything else manually. The hard part was re-syncing those tasks whenever a schedule is edited without losing a task’s “done today” state or its completion history (care_task_logs): FeedingTaskPlanner matches an existing reminder to a schedule row by its stored time, so renaming a meal label or changing its portion updates the same care_tasks row, while a removed meal removes only its own reminder. This logic is covered by FeedingTaskPlannerTest, which the team could run without an emulator because the planner has no Android imports.

(c) Distinguishing a one-off symptom from a pattern. A rule that fires on any single matching answer would be noisy (a cat can skip one meal for ordinary reasons), while a rule that waits for a long fixed streak would miss owners who only log a few days a week. HealthRuleEvaluator supports two rule shapes: fixed-window signal rules (“seen on at least N of the last W days”) for symptoms with an obvious threshold, and baseline-ratio rules that compare a recent logging rate against the cat’s own longer prior rate, which only fire once enough days have been logged in both windows. A rule can also require a minimum weight drop over a lookback period, computed by WeightTrend from the weight log. The team considered and rejected a simpler “any one bad day” rule set early on (noted directly in the rule-matching code’s “any”/“all” match-mode handling, which remains for simpler signals where a single clear symptom is enough) in favour of this two-shape design, at the cost of a more complex rule-authoring format in health_rules.json.

(d) Reaching for a non-permitted library when the permitted toolchain was insufficient. The team’s technical reference explicitly excludes Glide, instructing the use of hand-written bitmap sampling instead. Loading both local diary photographs (which must be decoded at a real, sharp size rather than a small cached preview) and remote teaser images for Reading Corner articles (an og:image URL resolved once and stored in seed data) without Glide would have meant re-implementing disk caching, down-sampling, and lifecycle-aware request cancellation twice. The team added Glide as a single, documented exception rather than hand-rolling a second image pipeline; this exception is not yet recorded in AUDIT_REPORT.md’s “outside course content” table alongside the earlier-approved exceptions (TextureView, the Application subclass, and the Activity Result API), which is flagged in REPORT_AUDIT.md as something the team should explicitly ratify before submission.

(e) Scraping a third-party website safely at runtime. The “more info” feature needed real supporting reading for an alert, but the team had no content-management backend of its own and was not permitted to add Retrofit/OkHttp. HealthArticleClient therefore issues plain HttpURLConnection GET requests against icatcare.org, with a capped response size, a request timeout, and — specifically to avoid the client silently following a redirect to an unrelated host — a check that the final response host is icatcare.org or a subdomain of it before any content is used. The alternative of bundling a fixed, offline set of articles was rejected because it could not reflect the specific signs behind whichever alert fired; the live-search approach’s limitation, discussed further in Section 3.4, is that it depends on a page layout the team does not control.

3.2 User interaction and usability

The Daily tab is designed to be the one screen an owner needs to open most days: the cat’s status, today’s check-in, and the day’s care tasks are all visible without navigating away, and the check-in itself is answered with tappable chips rather than free text, which keeps a daily log to a few seconds for a typical day. The decision-tree structure means a chip that needs more detail (for example, an unusual litter-box observation) only

asks its follow-up question when that chip is actually selected, so a routine day's check-in stays short while an unusual day can still be logged in more detail. Reopening the check-in later the same day restores the saved answers rather than starting over, which matters for a habit the app wants to be reopened after a short interruption. The bottom navigation bar keeps the three other regular destinations (Tasks, Diary, Profile) one tap away at all times, following the standard Android bottom-navigation pattern for a small number of top-level destinations [6]. The Reading Corner and health-alert banner are both derived from data the owner already logged rather than requiring a separate action, which was a deliberate choice to avoid adding a second thing to remember to check.

3.3 Comparison with existing applications

Functional differences are summarised against the three reference products from Section 1.2. Litterboxd's check-in screen follows Flo's chip-based daily-logging pattern [9] applied to a different domain (a pet's observable state rather than a menstrual cycle), and its Diary tab follows Letterboxd's dated-entry model [8] but, unlike Letterboxd, interleaves the owner's own entries with system-generated care-history lines rather than logging only one content type. Unlike 11pets, whose public description centres on reminders and vet-record sharing without describing any pattern detection across logged days [10], Litterboxd's alerts are explicitly derived from the comparison of a recent window against the cat's own history, not from a fixed reminder schedule. The team makes no claim that this makes Litterboxd better for an owner in practice — no usability study was run — only that the specific combination of a dated personal diary, a daily check-in, and history-aware alerting does not appear together in any one of these three reference products as documented.

3.4 Professional, legal, social, and ethical considerations

Data storage and security. All cat data (profiles, weights, check-in answers, diary entries, and photographs) is stored only in the app's own private SQLite database and internal storage, with allowBackup enabled via the standard Android auto-backup mechanism; nothing is transmitted to a server the team operates, because the application has no backend of its own. This limits exposure compared to a cloud-synced product, but it also means there is no cross-device backup beyond Android's own backup service, and no account recovery if the device is lost.

Third-party network access. The only outbound network traffic the app generates is the live, read-only search against icatcare.org described in Section 3.1(e). No personal or cat-identifying data is sent in that request beyond the search terms derived from the alert that fired; no user account, authentication, or persistent identifier is transmitted. This is a dependency on a third party's website remaining available and structured as expected, which is an accepted limitation rather than a resolved one — a page-layout change on icatcare.org would silently reduce the feature to "no matching articles" rather than fail loudly, since HealthArticleClient's find() method returns an empty list on any IOException.

Runtime permissions. POST_NOTIFICATIONS is requested once from HomeActivity and its denial is handled (a fix introduced during the audit pass); location permissions are declared in the manifest for a future "nearby vet clinics" feature but are never requested or used by any current Activity, consistent with the principle of not asking for a permission before the feature that needs it exists.

Responsible presentation of health information. The health-alerts feature is explicitly worded as "worth watching" or "see a vet" rather than as a diagnosis, and its "more info" pages are kept to pages that textually mention the same signs that triggered the alert. This reduces, but does not eliminate, the risk that an owner could over-interpret a pattern-matched alert as professional veterinary guidance; the application displays no disclaimer to that effect in the UI today, which is a gap the team should close before any wider release.

Data retention and deletion. Deleting a cat cascades to its weight logs and feeding schedule through SQLite foreign keys with ON DELETE CASCADE (verified by an instrumented test added during the audit pass), and also deletes its photo file. A full app reset deletes all rows and photo files and clears SharedPreferences. There is, by design, no server-side copy to separately delete.

3.5 Limitations

Fully implemented: onboarding and multi-cat profiles; feeding-schedule-derived and manual care tasks with AlarmManager reminders; the daily check-in decision tree with history-aware follow-ups; the baseline/signal/weight-trend health-alert engine; the diary (memories, milestones, and merged care history) with photo support; local weight charting.

Partially implemented: the live "more info" search depends entirely on icatcare.org's current page structure and returns nothing (rather than a clear error) if that structure changes or the device is offline; the Reading Corner's remote teaser images are resolved once into static seed data rather than refreshed from a live feed.

Not implemented: there is no account system, no cloud sync, and no social or sharing feature of any kind; the location permissions declared in the manifest support no feature yet; CSV export exists in FileStorageHelper but is not wired to any screen.

Known technical limitations: the app has no backend, so health rules and article content ship inside the APK (res/raw/*.json) and can only be updated by a new release; the health-alert engine's baseline comparison needs one to two weeks of logged history before it can fire at all, which was a deliberate design trade-off (Section 3.1c) but means a brand-new user sees no pattern-based alerts regardless of how the cat is actually doing.

4. Conclusion

Litterboxd is a Java Android application for daily cat care that combines a chip-based daily check-in, a rule-based health-alert engine that compares a cat's recent logged answers against its

own history rather than a fixed threshold, automatically-synchronised feeding reminders, and a personal diary that merges owner-written memories with care history read from the same database. The team built this on a deliberately constrained toolchain (plain SQLite, AlarmManager, and ExecutorService, with a single documented exception for Glide image loading) and, before layering on most of the above, ran and closed out a twenty-four-item internal audit of the original onboarding and persistence code, which is itself part of what this report is reporting on.

What the team learned was less about any individual Android API and more about the cost of deferred correctness work: the audit phase took as long as it did specifically because validation, rotation handling, and thread-safety had been skipped during the first onboarding implementation, and fixing them afterward required touching nearly every screen written up to that point. The health-rule and decision-tree engines, by contrast, were written test-first as plain Java from the start and needed almost no later rework, which the team takes as a point in favour of pushing logic out of Activities wherever an Android dependency is not actually required.

The application's clearest current limitations are its complete dependence on a third-party website for supplementary reading content, the one-to-two-week delay before any history-based health alert can fire for a new cat, and the absence of any account, backup, or sharing mechanism beyond what Android itself provides. Future work identified during this audit, in order of likely value, would be: ratifying and documenting the Glide exception in the project's own technical reference; adding an explicit non-diagnostic disclaimer to the health-alerts screen; and wiring the existing but unused CSV export into the Profile tab so an owner can hand a vet a readable history without screenshotting the app.

5. References

[1] I. Li, A. K. Dey, and J. Forlizzi, "A stage-based model of personal informatics systems," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '10), Atlanta, GA, USA, 2010, pp. 557–566, doi: 10.1145/1753326.1753409.

[2] J. Rooksby, M. Rost, A. Morrison, and M. Chalmers, "Personal tracking as lived informatics," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '14), Toronto, Canada, 2014, pp. 1163–1172, doi: 10.1145/2556288.2557039.

[3] Android Developers, "Save data using SQLite," Google. [Online]. Available: https://developer.android.com/training/data-storage/sqlite

[4] Android Developers, "Schedule repeating alarms," Google. [Online]. Available: https://developer.android.com/develop/background-work/services/alarms/schedule

[5] Android Developers, "Create a notification," Google. [Online]. Available: https://developer.android.com/develop/ui/views/notifications/build-notification

[6] Android Developers, "Request runtime permissions," Google. [Online]. Available: https://developer.android.com/training/permissions/requesting

[7] Android Developers, "Understand the Activity lifecycle," Google. [Online]. Available: https://developer.android.com/guide/components/activities/activity-lifecycle

[8] Letterboxd Limited, "About Letterboxd — Frequently Asked Questions." [Online]. Available: https://letterboxd.com/about/faq/

[9] Flo Health Inc., "How do I use the app?" Flo Help Center. [Online]. Available: https://help.flo.health/hc/en-us/articles/360014347632-How-do-I-use-the-app

[10] 11pets, "11pets: Pet Care," App Store product listing. [Online]. Available: https://apps.apple.com/us/app/11pets-pet-care/id1232470530

[11] Bumptech, "Glide: an image loading and caching library for Android," GitHub repository. [Online]. Available: https://github.com/bumptech/glide